

Overview of the project

- Objective: efficient (and correct!) Implementation of an algorithm
- The source code will be uploaded to a server, compiled and run automatically
- Deadlines: 8 September
- Project execution:
 - implementation in the C language
 - only with standard library (libc)
 - no threads or parallelization techniques
 - inputs and outputs on stdin and stdout
- The program must compile and run correctly
- Public Test cases will be made available as evidence to check for correct operation of the program
- After loading on the server, the program is run on tests, divided into 2 parts: public and private
- The correctness (output results) and efficiency (response times and memory occupied) of the program are measured

Plagiarism:

- Project to be carried out individually and in total autonomy (not in groups)
- Do not upload it to public repositories and don't pass it on for "inspiration" to colleagues
- Automated plagiarism control
- In case of copying all the projects involved are canceled

The Project:

A text editor with undo and redo, with the following commands:

- ind1, ind2c “change the text in the lines between ind1 and ind2 (ind2-ind1 +1 lines followed by a line with a single point)”
- ind1, ind2d “delete the lines between ind1 and ind2 (extremes included)”
- ind1, ind2p “print the lines between ind1 and ind2”
- nu “makes the undo of the last n commands (c or d)”
- nr “redo the last n undone commands with undo”
- q “terminate”

To pass the exam:

- There are 6 tasks. Each of these consists of 1 public subtask (i.e. with public expected input and output, 0 points) and 3 private subtasks of increasing difficulty (respectively 3+1+1 points)

- The maximum score obtainable is 30 points +1 extra point obtainable by passing a specific task
- To pass the exam it is necessary to through at least the simplest private subtask of each task and get a minimum score of 18/30
- The last implementation submitted in each task will be evaluated, you have infinite submissions until the deadline day
- The implementation evaluated must be identical for all tasks

Full Description

The project consists in implementing a simple text editor. The editor considers a document as a sequence of lines, of arbitrary size, numbered starting from one.

The editor interface is made up of text commands, terminated by a "newline" character. The commands can be followed by a portion of text, consisting of one or more lines, terminated by a character "." (period) which appears as the only character on the next line. The commands consist of a single letter, optionally preceded by one or two whole numbers.

The editor interface is freely inspired by that of the traditional editor "ed".

In some commands, the integers that appear in them represent address specifiers. More precisely, an address specifier is a number n , expressed in decimal, which indicates the address of the n -th line; the first line of the text has address 1.

The supported commands are the following, with the convention that $ind1, ind2$ indicate two address specifiers such that $ind1 \leq ind2$ and round brackets are introduced for ease of reading this text, but not included in the command:

- $(ind1, ind2) c$

Change the text present on the lines between $ind1$ and $ind2$ (extremes included). The text following the command must be made up of a number of lines equal to $ind2 - ind1 + 1$.

$ind1$ must be either an address actually present in the text, or the first address after the last line in the text (or 1 if the text is still empty).

- $(ind1, ind2) d$

Delete the lines between $ind1$ and $ind2$ (extremes included), moving up the lines following that of address $ind2$ (if there are any). Deleting a line that does not exist in the text has no effect.

- (ind1, ind2) p

Print the lines between ind1 and ind2 (included). Where there is no line in the text in the position to be printed, a line is printed containing only the period character "." followed by a "newline" character.

- (number) u

Undo a number of commands (c or d) equal to that specified in round brackets (where number is an integer strictly greater than zero). A sequence of consecutive undo commands cancels a number of steps equal to the sum of the steps specified in each of them. If the number of commands to be canceled is higher than the number of executed commands, all steps are canceled.

The execution of a text modification command (c, d) after an undo cancels the effects of the definitively canceled commands.

Note that the commands that have no effect are also counted in the number of commands to be canceled (for example, the cancellation of a block of lines that do not exist).

- (number) r

It cancels the effect of undo for a number of commands equal to the number starting from the current version (redo function).

Note that number must be an integer strictly greater than zero. We therefore have a sequence of commands like

10u
5r

It is to all effects equivalent to the 5u command only. Similarly, the sequence

12u
3r
2u
7r

equivalent to the 4u command. In the event that the number of commands to be redone is greater than those currently canceled, the maximum number of redo possible is performed.

- q

Finish the execution of the editor

A line of text supplied to the editor can contain a maximum of 1024 characters.

Assume that only correct commands are given to the editor (it is therefore not necessary to verify their correctness). For example, a command c is never given in which $\text{ind1} > \text{ind2}$, or $\text{ind1} < 1$, or ind1 is not the address of a line in the text, nor is the address of the first line after the text .

Similarly for the other commands.

Note, however, that, in some cases, the command is allowed, but simply has no effect; for example, it is possible to give a command d in which ind1 is not the address of a line of text, but in this case the command has no effect on the text.

Task overview

Task	Name	Time limit	Memory limit	Type
Tutorial	Tutorial	1.000 second	256 MiB	Batch
WriteOnly	WriteOnly	7.400 seconds	3.00 GiB	Batch
BulkReads	BulkReads	1.250 seconds	240 MiB	Batch
TimeForAChange	TimeForAChange	2.100 seconds	540 MiB	Batch
RollingBack	RollingBack	2.100 seconds	550 MiB	Batch
AlteringHistory	AlteringHistory	2.100 seconds	400 MiB	Batch
Rollercoaster	Rollercoaster	2.700 seconds	1.03 GiB	Batch

Input example

1,2c
first line
second line
.
2,3c
new second line
third line
.
1,3p
1,1c
new first line
.
1,2p
2,2d
4,5p
1,3p
4,5d
1,4p
3u
1,6p
1r
1,3p
q

The input sequence above leads to the following output from the editor:

first line
new second line
third line
new first line
new second line
.
.
new first line
third line
.
new first line
third line
.

·
first line
new second line
third line

·
·
·
new first line
new second line
third line